

Introducción a Sensores Remotos y Procesamiento de Imágenes Satelitales con Google Earth Engine

Wilder Harvey Clavijo Beltrán / Maria Jose Duarte Hamón / Suli Vanesa Gonzáles Walt

Table of contents

1	Parte 1. Ficha Técnica de Insumos	1
2	Parte 2. Diccionario de funciones GEE	3
3	Parte 3. Evidencias de integración GIS	9
3.1	Sentinel 2	9
3.2	Sentinel 1	11
3.3	MODIS	13
3.4	STRM	15
3.5	NDVI	17
4	Preguntas teoricas:	18
4.1	Sobre arquitectura cliente-servidor: Explica con tus propias palabras el flujo de información cuando ejecutas un filtro en GEE. ¿Por qué tu computadora portátil no colapsa ni se queda sin memoria RAM al buscar, filtrar y calcular el NDVI de una colección de miles de imágenes Sentinel-2? Haz referencia al uso de la función <code>.serialize()</code>	18
4.2	Sobre sensores crudos vs. productos derivados: Contrasta la naturaleza de la colección COPERNICUS/S2_SR_HARMONIZED frente a la colección WorldPop/GP/100m/pop. ¿Qué diferencia fundamental existe entre lo que capta el lente de un satélite y cómo se obtiene un mapa de habitantes por píxel?	18
4.3	Sobre álgebra de mapas: Explica físicamente por qué la fórmula del NDVI () arroja valores cercanos a 0.8 en el Parque El Salitre, pero arroja valores negativos o cercanos a cero sobre áreas completamente urbanas.	18

1 Parte 1. Ficha Técnica de Insumos

Identificador	Satélite/Sensor	Resolución espacial	Resolución temporal	Resolución espectral	Resolución radiométrica	Productos derivados
LANDSAT/LC09/C02/T19 TOA-SAT	Raster Landsat-OLI-2 / TIRS-2	30 m (visible, NIR, SWIR), 100 m (TIR, remuestreado a 30 m)	16 días	Visible (B1–B4), NIR (B5), SWIR (B6–B7), TIR (B10–B11)	12 bits	No aplica
COPERNICUS/S2_SR_HARMONIZED	Raster Sentinel-2 / MSI	10 m (visible, NIR), 20 m (SWIR), 60 m (aerosoles y vapor de agua)	5 días	Aerosoles (B1), visible (B2–B4), NIR (B8), SWIR (B11–B12)	12 bits	No aplica
COPERNICUS/S1_GRD	Raster Sentinel-1 / SAR	10 m	6 días	Polarizaciones (VV, VH)	16 bits	No aplica
MODIS/061/MOD13A2	Raster MODIS	500 m	8 días	Bandas de reflectancia (B1–B7)	12 bits	No aplica
USGS/SRTMGL1_003	Raster SRTM	30 m	Estático	Elevación	16 bits	Modelo digital de elevación global derivado de interferometría radar
UCSB-CHG/CHIRPS/DAILY	Raster CHIRPS	~5.6 km	1 día	Precipitación	No especificado	Combina datos de estaciones terrestres con imágenes satelitales infrarrojas
GOOGLE/Reese-OpenBuildings/v4/buildings	Reese-OpenBuildings	No aplica (derivado de imágenes de alta resolución ~50 cm)	Estático	No aplica	No aplica	Huellas de edificios generadas mediante modelos de inteligencia artificial
NOAA/VIIRS/DNB/MONTHLY_V1/VCMSLCFG	Raster VIIRS Black Marble	~463.8 m	1 mes	Radiancia nocturna (avg_rad), cobertura sin nubes (cf_cvg)	14 bits	Radiancia nocturna promedio mensual
OPERA/DSASXSL3inV1/S1	Raster Sentinel-1 (DSWx)	30 m	6–12 días	Clasificación de agua (WTR), mapa binario agua/no agua	No especificado	Detección de cuerpos de agua
WorldPop/100m/Pop	Raster WorldPop	~100 m	Anual	Población	No aplica	Estimación de población por celda

2 Parte 2. Diccionario de funciones GEE

1. `ee.Geometry.Point()`

La función `ee.Geometry.Point()` pertenece al módulo de geometrías de la API de Google Earth Engine y se utiliza para crear un objeto geométrico puntual que representa una localización específica en el espacio geográfico, es una clase fundamental dentro del modelo de datos vectoriales de Earth Engine y sirve como base para operaciones espaciales, filtrado de imágenes, recorte de datos raster y cálculos regionales.

Internamente, un objeto `Geometry` en GEE se almacena como una estructura geoespacial en el servidor, por lo que su creación no genera inmediatamente datos en el cliente, sino una referencia a una geometría que será evaluada cuando se ejecute una operación.

El constructor `Point()` recibe como parámetro principal una lista de coordenadas en formato [longitud, latitud], expresadas en grados decimales bajo el sistema de referencia geográfico WGS84 (EPSG:4326), que es el sistema por defecto en Earth Engine. Opcionalmente, se puede especificar un sistema de referencia espacial diferente mediante parámetros adicionales.

Parámetros principales:

`coords`: lista o tupla con coordenadas [lon, lat] `proj` (opcional): sistema de referencia espacial geodesic (opcional): define si la geometría se interpreta sobre la superficie esférica

Este tipo de objeto se utiliza comúnmente como región de interés (ROI), punto de muestreo, ubicación de sensores, estaciones meteorológicas o centros de parcelas experimentales.

2. `ee.FeatureCollection()`

La función `ee.FeatureCollection()` crea una colección de entidades geográficas vectoriales, donde cada elemento es un objeto `ee.Feature` compuesto por una geometría y un conjunto de atributos. Esta estructura corresponde al equivalente de una capa vectorial en SIG, similar a un shapefile o una tabla espacial.

En Earth Engine, las colecciones de tipo `FeatureCollection` son utilizadas para representar límites administrativos, polígonos de parcelas, puntos de muestreo, rutas, zonas de estudio o cualquier otro conjunto de objetos espaciales con información asociada.

El constructor puede recibir diferentes tipos de entrada:

ID de un asset almacenado en Earth Engine lista de features otra colección existente geometrías convertidas en features

Los objetos `FeatureCollection` se almacenan en el servidor y permiten realizar operaciones espaciales como intersección, filtrado, selección de atributos, reducción estadística y recorte de imágenes.

Parámetros principales:

`tableId`: identificador de asset lista de `ee.Feature` objeto `ee.Geometry` convertido a feature

3. `ee.ImageCollection()`

La clase `ee.ImageCollection()` representa una colección de imágenes raster organizadas como una serie de datos multidimensionales. Cada elemento de la colección es un objeto `ee.Image`, y las colecciones suelen corresponder a datasets satelitales multitemporales como Sentinel-2, Landsat, MODIS o colecciones climáticas.

En Earth Engine, las colecciones no se descargan al cliente, sino que se manejan como objetos de servidor, lo que permite procesar grandes volúmenes de datos mediante evaluación diferida (lazy evaluation).

El constructor recibe como argumento el identificador del dataset, el cual corresponde a una ruta dentro del catálogo público o a un asset privado.

Parámetros:

collectionId: ID del dataset Las colecciones permiten aplicar operaciones encadenadas como:

filtros espaciales filtros temporales filtros por atributos transformaciones espectrales reducción estadística

4. ee.Image()

La función ee.Image() crea o referencia una imagen raster individual dentro del entorno de Earth Engine. Un objeto Image puede provenir de una colección, de un asset, o de una operación matemática.

Las imágenes en GEE son estructuras multibanda que pueden contener información espectral, térmica, radar o índices derivados. Cada banda se almacena como un raster independiente dentro del mismo objeto.

Parámetros posibles:

ID de imagen número constante resultado de operación otra imagen

Las imágenes son la base para cálculos espectrales, clasificación, índices de vegetación, análisis temporal y estadísticas espaciales.

5. .filterBounds()

El método .filterBounds() se aplica sobre colecciones (ImageCollection o FeatureCollection) y permite filtrar los elementos que intersectan una geometría dada.

La operación se ejecuta en el servidor y forma parte del modelo de evaluación diferida de Earth Engine, por lo que no produce resultados inmediatos, sino que modifica la definición de la colección para que las operaciones posteriores se realicen únicamente sobre los datos que cumplen la condición espacial.

El parámetro principal de esta función es un objeto de tipo ee.Geometry o ee.FeatureCollection, el cual define el área de interés. Este método es fundamental para reducir el volumen de datos procesados y mejorar la eficiencia del análisis, especialmente cuando se trabaja con colecciones globales de imágenes satelitales.

6. .filterDate()

El método .filterDate() permite restringir una colección de imágenes a un intervalo temporal específico. Internamente, esta función compara la propiedad temporal de cada imagen, generalmente almacenada como system:time_start, con las fechas indicadas por el usuario.

Este filtro es esencial en análisis multitemporales, monitoreo de cambios, estudios fenológicos, evaluación de cultivos y cualquier aplicación en la que sea necesario trabajar con imágenes de un periodo determinado.

El método recibe como parámetros una fecha inicial y una fecha final, las cuales pueden expresarse como cadenas de texto, objetos de fecha o formatos compatibles con Earth Engine. La función no modifica los datos originales, sino que genera una nueva colección que contiene únicamente los elementos dentro del rango temporal especificado.

7. .filter(ee.Filter ...)

El método `.filter()` permite aplicar filtros personalizados a colecciones utilizando objetos de tipo `ee.Filter`. Este mecanismo proporciona mayor flexibilidad que los filtros espaciales o temporales, ya que permite evaluar atributos, comparar valores, combinar condiciones lógicas y seleccionar elementos en función de metadatos.

Los filtros pueden utilizar operadores de comparación como menor que, mayor que, igual a, distinto de, así como operadores lógicos como AND y OR. También es posible filtrar por propiedades específicas de las imágenes, como porcentaje de nubosidad, sensor, órbita o cualquier atributo almacenado en los metadatos.

El resultado de la operación es una nueva colección que contiene únicamente los elementos que cumplen la condición especificada. Esta función es ampliamente utilizada para depurar datasets, seleccionar imágenes de mejor calidad y preparar colecciones antes de realizar cálculos espectrales o estadísticos.

8. `.sort()`

El método `.sort()` se utiliza para ordenar los elementos de una colección según el valor de una propiedad determinada. Este ordenamiento puede realizarse en sentido ascendente o descendente, dependiendo del criterio definido por el usuario.

La función es especialmente útil cuando se requiere seleccionar una imagen con características específicas, como la menor nubosidad, la fecha más reciente o el menor error de medición. El ordenamiento se realiza en el servidor y no modifica los datos originales, sino que crea una nueva colección con los elementos reorganizados.

El parámetro principal es el nombre de la propiedad por la cual se desea ordenar, y opcionalmente se puede indicar el sentido del orden.

9. `.first()`

El método `.first()` devuelve el primer elemento de una colección después de haber aplicado filtros, ordenamientos u otras operaciones. El resultado es un único objeto, que puede ser una imagen o una feature, dependiendo del tipo de colección.

Este método se utiliza comúnmente después de aplicar `.sort()` para seleccionar el elemento que cumple mejor un criterio determinado, por ejemplo, la imagen con menor porcentaje de nubes o la primera escena disponible dentro de un rango temporal.

La operación se ejecuta en el servidor y devuelve una referencia al elemento seleccionado, sin descargar los datos al cliente.

10. `.clip()`

El método `.clip()` se utiliza para recortar una imagen utilizando una geometría o una colección de features como límite espacial. El resultado es una nueva imagen en la que únicamente se conservan los píxeles que se encuentran dentro de la región especificada.

Esta operación no modifica la imagen original, sino que crea una nueva instancia con los valores restringidos al área de interés. El recorte se realiza en el servidor y es una práctica común para reducir el tamaño de los datos antes de realizar cálculos estadísticos, exportaciones o visualizaciones.

El parámetro principal es un objeto geométrico que define el límite espacial del recorte.

11. `.normalizedDifference()`

El método `.normalizedDifference()` calcula la diferencia normalizada entre dos bandas de una imagen, utilizando una operación matemática que produce valores normalizados entre -1 y 1 . Este tipo de cálculo es ampliamente utilizado en teledetección para generar índices espectrales como NDVI, NDWI o NDBI.

La función recibe como parámetro una lista con los nombres de las dos bandas que se desean comparar. Internamente, Earth Engine aplica la fórmula de diferencia normalizada píxel por píxel y genera una nueva imagen con el resultado.

Este método se ejecuta completamente en el servidor y permite trabajar con grandes imágenes sin necesidad de transferir datos al cliente.

12. `.getInfo()`

El método `.getInfo()` se utiliza para transferir información desde el servidor de Earth Engine hacia el cliente. Debido al modelo de ejecución diferida, la mayoría de los objetos en GEE son solo referencias hasta que se solicita su evaluación, y `.getInfo()` fuerza esa evaluación.

El resultado se devuelve en formato JSON, lo que permite inspeccionar valores, imprimir resultados o utilizarlos en el código Python. Sin embargo, esta función debe usarse con precaución, ya que puede generar tiempos de espera elevados si se intenta descargar grandes cantidades de datos.

Su uso se recomienda únicamente para depuración, verificación de resultados o extracción de valores pequeños.

13. `.serialize()`

El método `.serialize()` convierte un objeto de Earth Engine en una representación estructurada en formato JSON. Esta función se utiliza principalmente para depuración, almacenamiento de configuraciones o análisis interno del objeto.

La serialización permite visualizar la estructura completa de la operación que será ejecutada en el servidor, incluyendo filtros, transformaciones y dependencias. Esto resulta útil para comprender cómo Earth Engine interpreta la cadena de operaciones definida en el código.

El resultado es un texto estructurado que describe el objeto, pero no contiene los datos raster reales.

14. `.reduceRegion()`

El método `.reduceRegion()` se utiliza para aplicar un reductor estadístico sobre los valores de los píxeles de una imagen dentro de una región definida. Esta función permite obtener estadísticas como promedio, mínimo, máximo, suma o desviación estándar dentro de un área específica.

El método requiere especificar un objeto reductor, una geometría, una escala espacial y un número máximo de píxeles permitidos para el cálculo. La operación se realiza en el servidor y devuelve un diccionario con los resultados.

Este método es fundamental en análisis cuantitativos, validación de índices espectrales, estudios agronómicos, monitoreo ambiental y evaluación de variables biofísicas.

15. `ee.Reducer.mean()`

El objeto `ee.Reducer.mean()` define un reductor estadístico que calcula el valor promedio de los píxeles dentro de una región. Se utiliza junto con funciones como `reduceRegion()` para obtener estadísticas resumidas de una imagen.

El cálculo se realiza sobre todos los píxeles que se encuentran dentro de la geometría especificada y que cumplen las condiciones definidas en la escala y el número máximo de píxeles.

Este reductor es uno de los más utilizados en análisis de teledetección, ya que permite obtener valores representativos de índices espectrales, temperatura, reflectancia o cualquier variable raster.

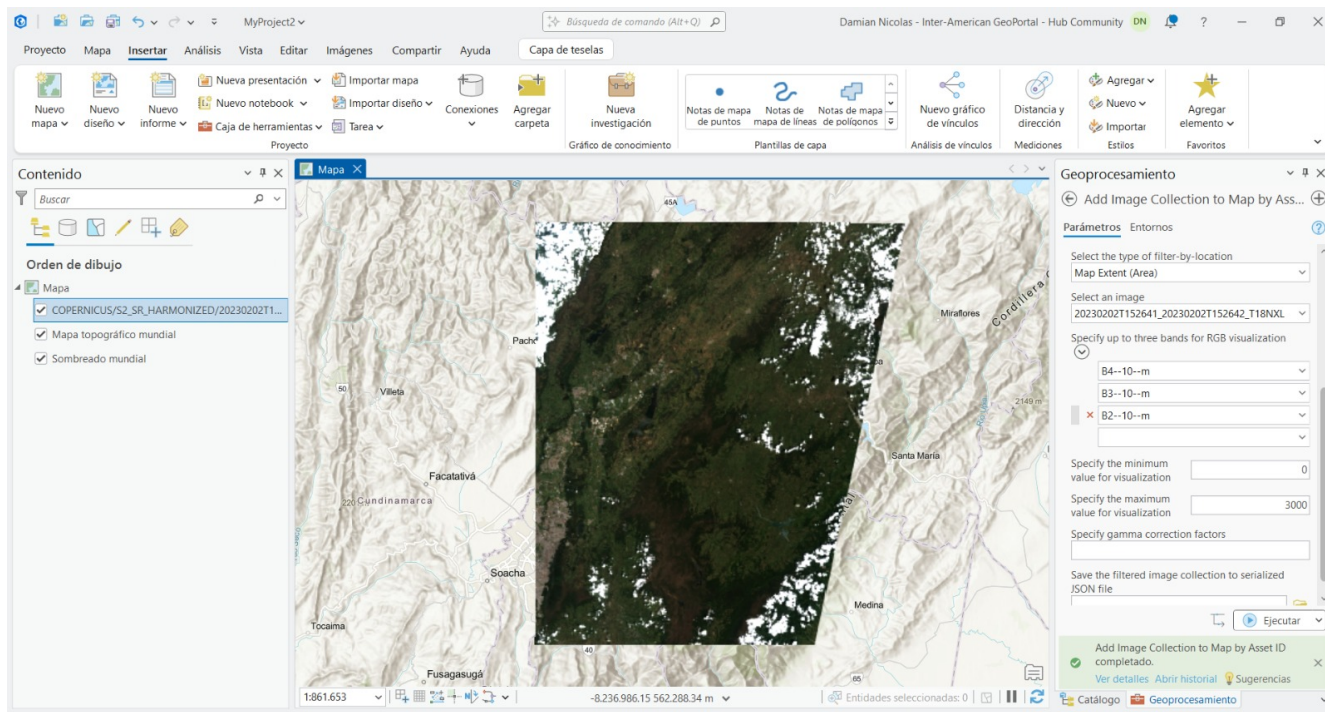
16. `ee.Reducer.minMax()`

El objeto `ee.Reducer.minMax()` define un reductor que calcula simultáneamente el valor mínimo y el valor máximo de los píxeles dentro de una región. Este tipo de reducción es útil para conocer el rango de valores de una variable, detectar valores extremos o verificar la calidad de los datos.

El resultado se devuelve como un diccionario que contiene ambos valores para cada banda analizada. La operación se ejecuta completamente en el servidor y se utiliza frecuentemente en análisis exploratorios, validación de resultados y preparación de visualizaciones.

3 Parte 3. Evidencias de integración GIS

3.1 Sentinel 2



Geoprocresamiento

← Add Image Collection to Map by Ass... →

Parámetros Entornos

Specify the image collection asset ID

COPERNICUS/S2_SR_HARMONIZED

Filter by properties

Property Name PIXEL_PERCENTAGE

Operator <

Filter Value 10

+ Agregar otro

Filter by dates

Starting Date

Ending Date

2023-01-01

2023-12-31

Geoprocresamiento

← Add Image Collection to Map by Ass... →

Parámetros Entornos

2023-01-01

Select the type of filter-by-location

Map Extent (Area)

Select an image

20230202T152641_20230202T152642_T18NXL

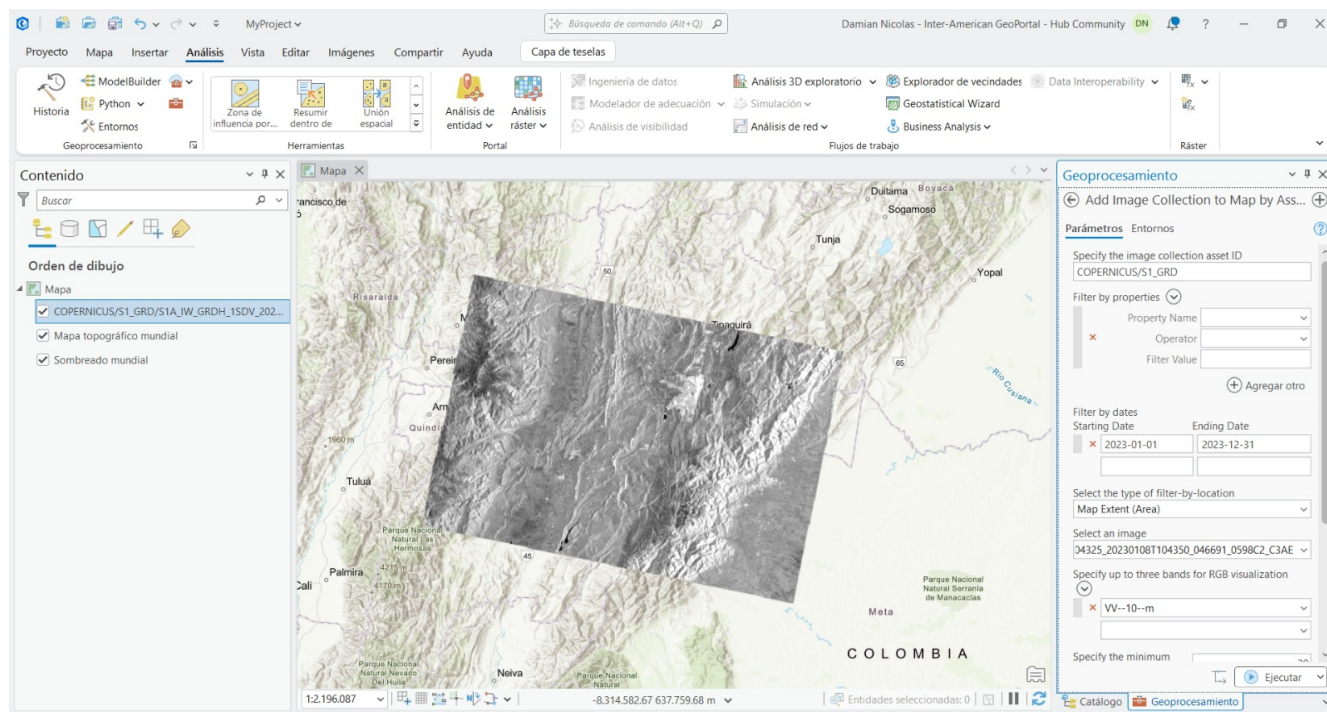
i Specify up to three bands for RGB visualization

B4--10--m

B3--10--m

B2--10--m

3.2 Sentinel 1



Geoprocresamiento

← Add Image Collection to Map by Ass... →

Parámetros Entornos

Specify the image collection asset ID

COPERNICUS/S1_GRD

Filter by properties

×

Property Name

Operator

Filter Value

+ Agregar otro

Filter by dates

Starting Date

Ending Date

×

2023-01-01

2023-12-31

Select the type of filter-by-location

11

Geoprocresamiento

← Add Image Collection to Map by Ass... →

Parámetros Entornos

×

2023-01-01

2023-12-31

Select the type of filter-by-location

Map Extent (Area)

Select an image

04325_20230108T104350_046691_0598C2_C3AE

Specify up to three bands for RGB visualization

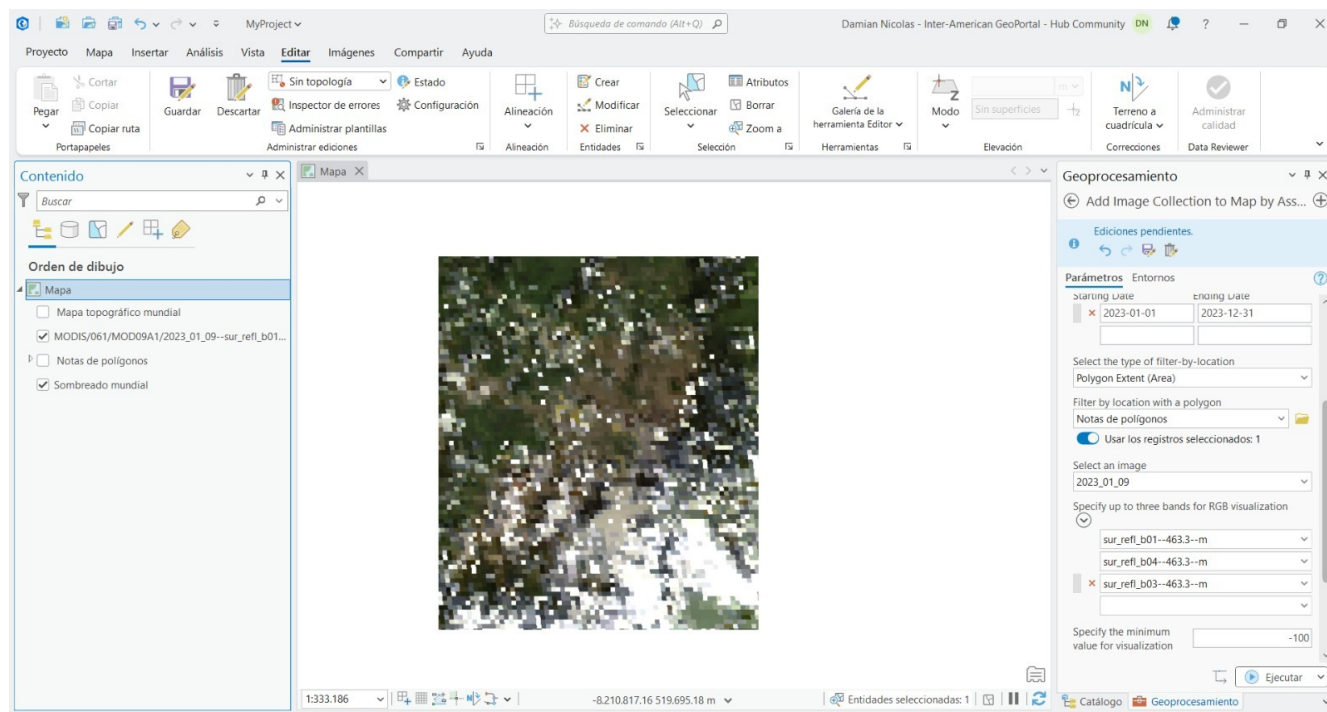
×

VV--10--m

Specify the minimum value for visualization

Specify the maximum value for visualization

3.3 MODIS



Geoprocésamiento

← Add Image Collection to Map by Ass... →

Ediciones pendientes.



Parámetros Entornos

Specify the image collection asset ID

MODIS/061/MOD09A1

Filter by properties

Property Name

Operator

Filter Value

+ Agregar otro

Filter by dates

Starting Date

Ending Date

2023-01-01

2023-12-31

Geoprocésamiento

← Add Image Collection to Map by Ass... →

Ediciones pendientes.



Parámetros Entornos

NOTAS DE POLÍGONOS

Usar los registros seleccionados

Select an image

2023_01_09

Specify up to three bands for RGB visualization

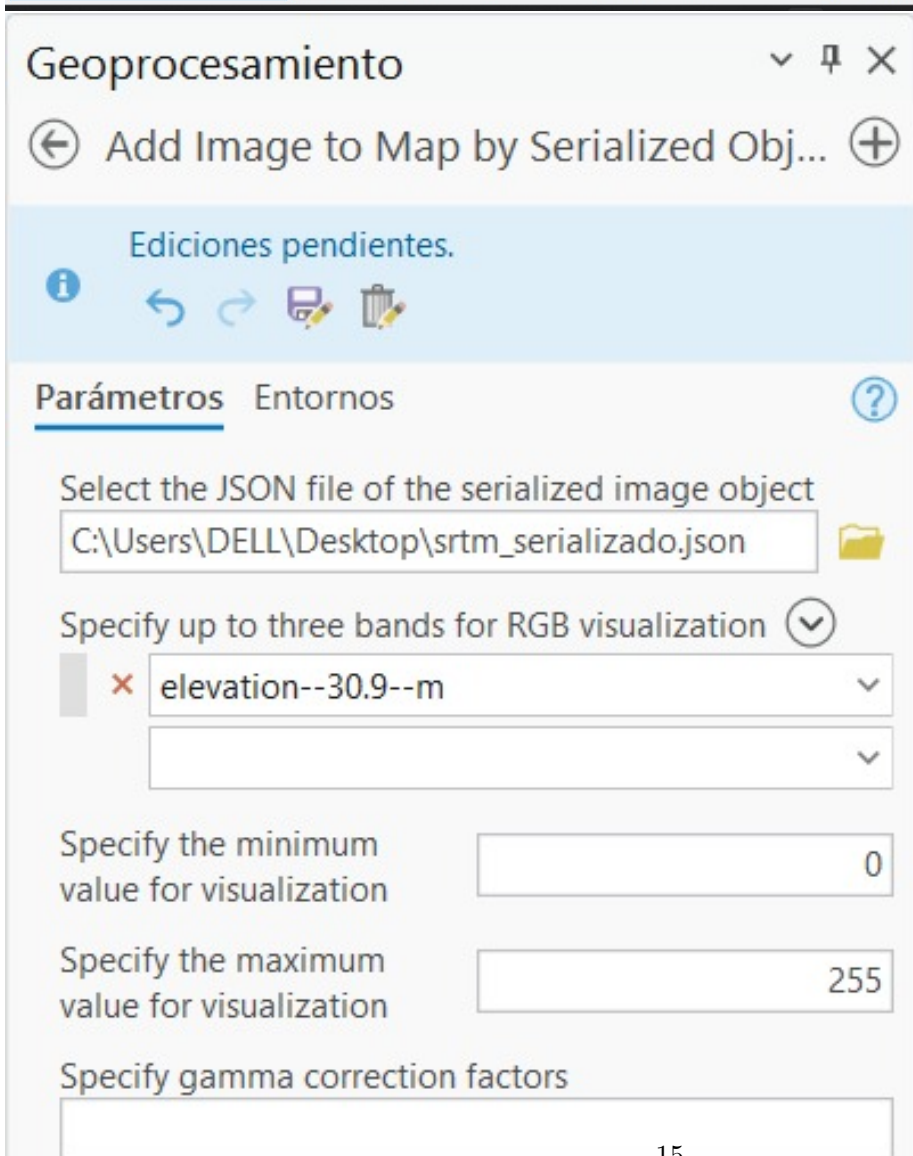
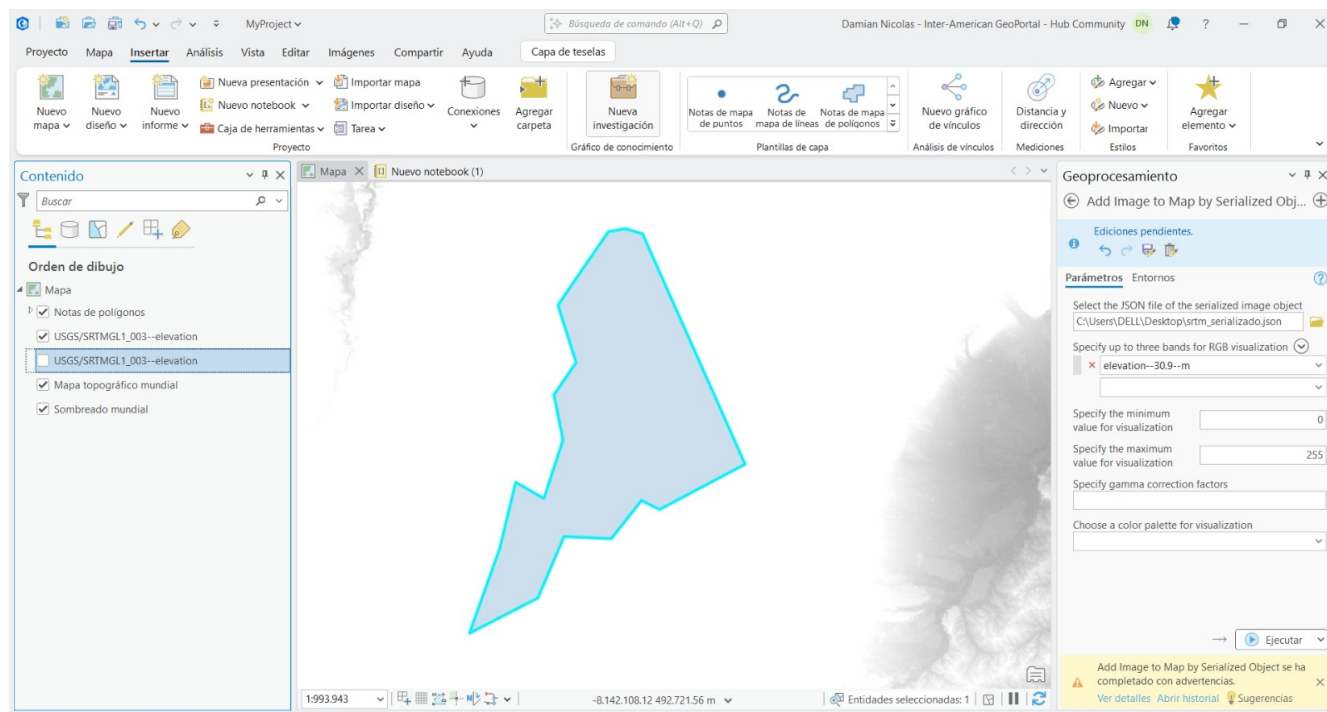
sur_refl_b01--463.3--m

sur_refl_b04--463.3--m

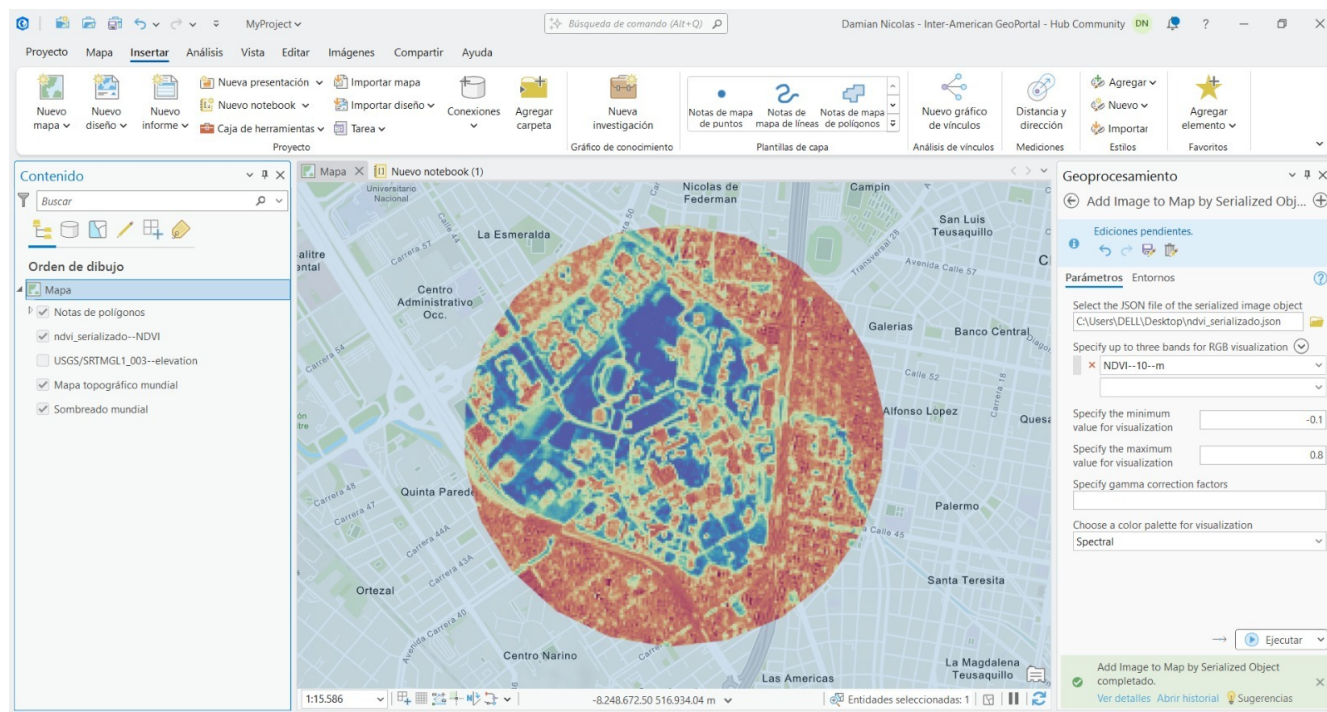
sur_refl_b03--463.3--m

Specify the minimum value for visualization

3.4 STRM



3.5 NDVI



Geoprocresamiento

← Add Image to Map by Serialized Obj... →

Ediciones pendientes.

Parámetros Entornos

Select the JSON file of the serialized image object

C:\Users\DELL\Desktop\ndvi_serializado.json

Specify up to three bands for RGB visualization

NDVI--10--m

Specify the minimum
value for visualization

-0.1

Specify the maximum
value for visualization

0.8

Specify gamma correction factors

4 Preguntas teoricas:

4.1 Sobre arquitectura cliente-servidor: Explica con tus propias palabras el flujo de información cuando ejecutas un filtro en GEE. ¿Por qué tu computadora portátil no colapsa ni se queda sin memoria RAM al buscar, filtrar y calcular el NDVI de una colección de miles de imágenes Sentinel-2? Haz referencia al uso de la función .serialize().

En Google Earth Engine, el flujo de información sigue un modelo cliente-servidor con evaluación diferida, donde el cliente es decir, la computadora, no procesa datos directamente, sino que construye un grafo computacional con las operaciones solicitadas, que incluyen filtrar imágenes Sentinel-2, calcular el NDVI, entre otras. Este grafo se convierte mediante la función .serialize() en un objeto JSON que describe el flujo de trabajo sin incluir los datos, y se envía a los servidores en los que se ejecuta de forma distribuida y paralela sobre grandes volúmenes de información. Por esta razón, la computadora no colapsa ni consume mucha memoria, ya que no descarga ni procesa las imágenes, solo actúa como intermediario enviando instrucciones y recibiendo los resultados.

4.2 Sobre sensores crudos vs. productos derivados: Contrasta la naturaleza de la colección COPERNICUS/S2_SR_HARMONIZED frente a la colección WorldPop/GP/100m/pop. ¿Qué diferencia fundamental existe entre lo que capta el lente de un satélite y cómo se obtiene un mapa de habitantes por píxel?

La colección COPERNICUS/S2_SR_HARMONIZED corresponde a un producto de sensores ópticos que registra la reflectancia de la superficie, es decir, una medida de la radiación electromagnética reflejada en distintas longitudes de onda tras ser corregida atmosféricamente, manteniendo una relación directa con las propiedades de la cobertura terrestre. Por otro lado, WorldPop/GP/100m/pop es un producto derivado, no proviene de una medición directa del sensor, sino de un modelo estadístico que integra datos censales con otras variables para estimar la distribución espacial de la población. Según esto, la diferencia es que el primero representa una variable física observada y el segundo es una variable inferida mediante modelación geoespacial.

4.3 Sobre álgebra de mapas: Explica físicamente por qué la fórmula del NDVI () arroja valores cercanos a 0.8 en el Parque El Salitre, pero arroja valores negativos o cercanos a cero sobre áreas completamente urbanas.

El índice de vegetación NDVI corresponde a la respuesta espectral de la vegetación, donde los pigmentos fotosintéticos absorben la radiación en la banda roja y reflejan en el infrarrojo cercano (NIR). En áreas con vegetación densa, como el Parque El Salitre, los valores de NIR superan a los de la banda roja, produciendo valores positivos altos (cerca de 0.8), lo que indica alto vigor vegetal; por el contrario, las superficies urbanas presentan respuestas espectrales más similares entre ambas bandas o mayor reflectancia en el rojo, lo que reduce el numerador de la ecuación y genera valores cercanos a cero o negativos.